Individual Project Progress Report

(June 8 – July 15)

(Name: Tanvir Ahmed Id: 2231047642)

Week 1 (Jun 8 – Jun 10)

Classes for CSE440.1 Artificial Intelligence had just started and the project team had not been formed yet. I reviewed the course syllabus and the semester project requirements, and read a little about how Markov Decision Processes are used for decision-making under uncertainty, so that I would be ready to contribute once the team and topic were decided.

Week 2 (Jun 15 – Jun 17)

The project team was formed with four members. As a group we discussed a few possible AI project ideas and settled on **Autonomous Volcanic Terrain Exploration Using a Markov Decision Process (MDP)**: an autonomous rover that must navigate a hazardous, randomly generated grid terrain (lava, craters, gas zones, rock obstacles) while balancing safety, exploration coverage, and science-point collection. We agreed an MDP was appropriate because the rover's movement is stochastic and its decisions are sequential, which fits the state/action/reward/policy structure of an MDP directly. From the start we cared about three competing goals – staying alive, covering ground, and collecting science – which later became the metrics I measure in my part.

Week 3 (Jun 22 – Jun 24)

The team opened a shared GitHub repository and split the work into four roles so that progress could be made in parallel: MDP core (terrain generation, state/action space, reward model, value iteration), agent & simulation, visualization & demo outputs, and experiments & comparison. I was assigned **Member 4 – Experiments and Comparison**, which owns `support/experiments.py`. My responsibility is to show *quantitatively* that the MDP explorer performs better than simple baseline strategies, and to produce the results table (`outputs/experiment_results.csv`) and the comparison plot (`outputs/performance_plot.png`) – without modifying the MDP, terrain, agent, or simulation logic that the other members own. We agreed on a one-branch-per-member Git workflow to keep everyone's work isolated until it is tested and merged.

Week 4 (Jun 29 – Jul 1)

Before writing any code I set up my environment (a Python virtual environment with `numpy`, `matplotlib`, and `pandas`) and read through the modules my part depends on, so my experiment would use the real interfaces instead of guessing at them:

- `support/terrain.py` – the `Terrain` class builds a seeded 10 × 10 grid; `get_cell(row,col)`, `is_valid_position()`, and `base_location` describe the environment.
- `support/mdp.py` – `VolcanicMDP` exposes `value_iteration()`, `extract_policy()`, `transition_probabilities()`, and `reward()`. Movement is stochastic: 0.75 intended, 0.10 slip-left, 0.10 slip-right, 0.05 stay.

At this point `agent.py` and `simulation.py` were still unfinished, so I planned `experiments.py` so it could drive `Terrain` and `VolcanicMDP` directly and later be re-connected to the finished agent. I designed the module around the six required functions (`run_mdp_agent`, `run_random_agent`, `run_greedy_agent`, `run_experiments`, `save_results`, `plot_results`) and defined the six metrics I would record for every run:

- **total reward** – sum of MDP rewards collected during the run;
- **coverage %** – distinct non-rock cells visited, over the total;
- **hazards entered** – number of moves into a lava/crater/gas cell;
- **science points** – distinct science cells collected;
- **survived** – whether the rover avoided a fatal lava cell;
- **total steps** – steps taken before the run stopped.

I then created my working branch `member4-experiments`.

Week 5 (Jul 6 – Jul 8)

I began implementing `support/experiments.py` with three agents that all share *one* environment, so the comparison is fair: they run on the same seeded terrain and sample the same stochastic movement model; only the way each chooses its next action differs.

- **MDP agent** – follows the value-iteration policy.
- **Random agent** – picks a random movement direction each step.
- **Greedy agent** – uses a breadth-first search to the nearest unvisited reachable cell (nearest-unvisited), treating rock and lava as impassable while planning.

One important detail I had to solve: the reward gives `SCIENCE` = +20 on *every* entry and `STAY` is a legal action, so the raw MDP policy makes the rover walk to the nearest science cell and stay there forever (on seed 42 this yields a reward of 1730 but only 6.1% coverage and a single science point). I fixed this at the experiment level with a *collect-once* rule – a gathered science cell becomes ordinary ground via `terrain.set_cell` – and had the MDP agent re-run value iteration after each collection so it keeps exploring toward new objectives instead of camping on one cell. Each run stops when the

Table 1: Average performance over 20 seeded terrains ($\gamma = 0.90$). The MDP wins on every metric except raw coverage, where the greedy baseline – a pure coverage maximizer – leads but pays for it in hazards and deaths.

Agent	Reward	Cov.%	Haz.	Sci.	Surv.%
MDP	**+42.0**	13.9	**1.9**	**3.5**	**60**
Greedy	−124.5	**29.3**	9.0	2.9	30
Random	−179.5	7.6	3.8	0.7	15

rover dies, reaches the step budget, or gets stuck in one place, so the three agents are always compared under the same rules.

Week 6 (Jul 13 – Jul 15) – Update 1 Complete

After Member 2 pushed the finished agent and simulation, I pulled that work into my branch (rebased `member4-experiments` onto the updated `main`) and integrated with it instead of duplicating logic:

- The random and greedy baselines became thin subclasses of `MDPExplorerAgent` that override only `choose_action`, so all three agents are measured by exactly the same inherited code (same movement sampling, reward, and hazard/science tracking).
- The discount factor is read from `data/terrain_config.json` ($\gamma = 0.90$). I found that because the MDP is risk-neutral it will accept a move with a 10% chance of slipping into lava over staying safe when the expected value is only marginally higher; a larger γ makes this worse (survival falls from $\sim$60% at 0.90 to $\sim$30% at 0.92), so using the documented config value matters.
- I ran all three agents over 20 seeds, wrote one row per run to `experiment_results.csv`, and saved the averaged bar-chart comparison to `performance_plot.png` (Table 1, Figure 1).
- I verified the results are deterministic (identical across repeated runs), that the script works from any directory, and that every module still runs; I also updated the README with a Week 5 "Experiments and Comparison" section, then committed and pushed my branch.
- I did *not* modify `terrain.py`, `mdp.py`, `agent.py`, or `simulation.py`, per the group's file-ownership rules – only their public methods are used.

Results and Discussion

Across 20 randomly generated terrains the MDP agent is the only one with a *positive* average reward (+42.0, versus −124.5 for greedy and −179.5 for random), enters the fewest hazards (1.9), collects the most science (3.5), and survives most often (60%). The greedy agent covers more ground (29.3%) because it is built purely to maximize coverage, but it walks straight through gas and craters, so it accumulates many hazards and dies twice as often as the MDP. The random agent is worst on almost every metric. This matches the project's goal: the MDP balances safety, science, and exploration, rather than chasing a single objective blindly.

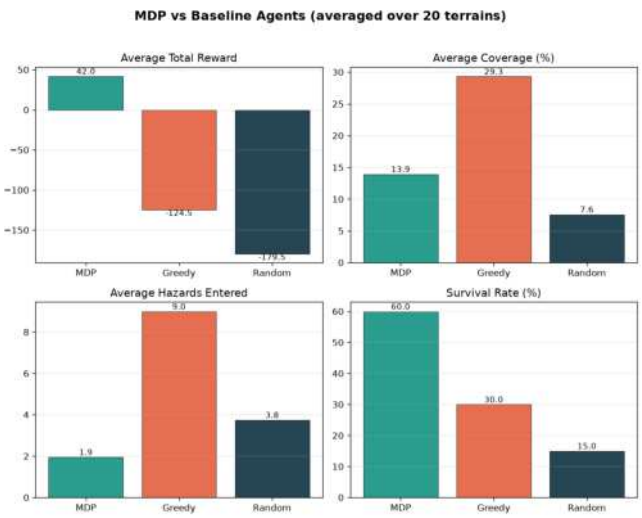

Figure 1: Final Update 1 output from `support/experiments.py`: average reward, coverage, hazards entered, and survival rate for the MDP agent versus the random and greedy baselines, over 20 seeds. The MDP is the only agent with positive reward, the fewest hazards, and the highest survival rate.

Testing and Verification

I ran the acceptance test from the project root before pushing, to confirm the module runs cleanly end to end and produces both output files:

```
$ python support/experiments.py
Agent Reward Coverage% Hazards ...
MDP 42.0 13.9 1.9 ...
Greedy -124.5 29.3 9.0 ...
Random -179.5 7.6 3.8 ...
Saved results CSV : outputs/experiment_results.csv
Saved performance : outputs/performance_plot.png
```

No exceptions were raised, one row was written per run (20 seeds × 3 agents = 60 rows), and both output files were produced on every run. Repeating the run gave byte-identical results, confirming the experiment is reproducible.

My Personal Contribution (Summary)

I set up my Python environment and reviewed the terrain and MDP interfaces my module depends on before writing any code. I implemented the full experiment-comparison module (`support/experiments.py`): three agents (MDP, random, and a breadth-first greedy) sharing one fair environment, the six required metrics, CSV export via `pandas`, and a `matplotlib` bar chart. I diagnosed and solved the science-"camping" problem with a collect-once plus re-plan rule, and read the discount factor from the project config after finding it strongly affects survival. After Member 2's simulation was merged, I rebased my branch, integrated by subclassing `MDPExplorerAgent` so all agents share identical measurement, ran the 20-seed comparison, and produced `experiment_results.csv` and `performance_plot.png`. I verified the numbers are reproducible and not hand-tuned, updated the README, and committed and pushed my own branch – without editing any file owned by another team member.